Universidad Nacional de Loja
Carrera en Computación
Facultad de Energía las Industrias y los Recursos Naturales no Renovables

Componente Autónomo

Diagramación de punteros y estados de memoria frente a la instanciación de objetos: Representaciones visuales del ciclo de creación, desde la asignación del puntero en el Stack hasta la reserva dinámica del objeto en el Heap.

Integrantes (Grupo 4)	Freddy Ordoñez, Yimmy Angulo, Joaquin Moscol, Erick Roguel, Santiago Guaman		
Paralelo	Tercer Ciclo		
Professor	Ing. Cristian Narváez G. Mg.Sc.		
Unidad	Unidad 1: Estructura de Datos y Gestión de Memoria		
Fecha	18/10/2026	**Uso de IA:** Si: No:X	**Calificación:**

Introducción

La diagramación de punteros y los estados de memoria frente a la instanciación de objetos son conceptos fundamentales para comprender cómo los lenguajes de programación gestionan los recursos de hardware, el almacenamiento de variables y la vida útil de los datos dinámicos.

1. Organización de la Memoria: Stack frente a Heap

La memoria *Stack* (pila) y la memoria *Heap* (montón) son las dos zonas principales que utiliza la memoria RAM de un ordenador para ejecutar un programa, pero funcionan de manera opuesta en cuanto a velocidad, tamaño y forma de gestión [1]:

- **El Stack (Pila de Ejecución):** Almacena los registros de activación (*activation records*) que se generan cada vez que ocurre una llamada a una función, conteniendo los parámetros pasados y las variables locales de duración automática [2]. Es una estructura organizada para datos estáticos cuya dimensión se conoce en tiempo de compilación, operando bajo un esquema LIFO (*Last-In, First-Out*) que garantiza un acceso extremadamente rápido.
- **El Heap (Montículo):** Es la región de memoria reservada desde donde se asignan objetos y bloques de datos de forma dinámica a medida que el programa lo solicita en tiempo de ejecución [4]. Está diseñada para datos cuyo volumen o vida útil cambian durante la ejecución, albergando tipos no primitivos como objetos, arreglos y funciones.
- **Liberación de Memoria (Garbage Collection):** A diferencia del Stack, donde las variables se limpian al salir de su contexto, la memoria del Heap en motores de JavaScript se gestiona y libera mediante el recolector de basura utilizando algoritmos de rastreo de objetos alcanzables [3].

2. Instanciación de Objetos y Punteros

En la arquitectura de memoria, la creación y gestión de tipos no primitivos se basa en la combinación armónica entre el Heap y el Stack [4]:

- **Instanciación en el Heap:** Ocurre cuando se crean estructuras complejas mediante objetos literales ({}), arreglos ([]), constructores con *new* o funciones. El motor reserva un bloque de memoria dinámica sin un límite fijo predeterminado [4].



- **Punteros y Referencias:** Una variable asignada a un objeto no guarda el objeto completo en el Stack, sino una referencia (puntero interno) que apunta directamente a la ubicación física del objeto alojado en el Heap [6].
- **Copia por Referencia y *Aliasing*:** Al asignar un objeto existente a una nueva variable ($let\ b = a$), no se duplica el contenido, sino que se copia la dirección de memoria, provocando que cualquier mutación a través de una variable se refleje inmediatamente en la otra [2].

3. Utilidad de los Diagramas de Estados de Memoria

Un diagrama de estado de memoria es una representación gráfica instantánea que muestra la distribución de las variables, valores y direcciones de memoria en un punto específico de la ejecución del programa [3]:

- **Comprensión del Ciclo de Vida:** Muestran de forma visual cómo se asignan y destruyen los marcos de activación en el Stack y los bloques dinámicos en el Heap [2].
- **Rastreo de Punteros:** Representan variables como cajas y referencias como flechas dirigidas, aclarando con precisión el comportamiento del *aliasing* [6].
- **Identificación de Fugas de Memoria (*Memory Leaks*):** Permiten detectar referencias huérfanas o no deseadas que impiden la liberación de recursos por parte del recolector de basura [5].

4. Análisis de Casos de Estudio y Diagramación de Estados de Memoria

En esta sección se analizan tres escenarios prácticos de gestión de memoria en JavaScript (V8 Engine), evaluando mediante diagramas cómo la instanciación y las referencias afectan al Stack y al Heap [5].

4.1. Caso de Estudio 1: Shared Lexical Context Leak (Fuga por Contexto Léxico)

Descripción del problema: Ocurre cuando una función interna retiene una referencia al ámbito léxico de su función padre a través de un cierre (*closure*), impidiendo que el recolector de basura libere objetos masivos en el Heap [5]. Este fenómeno se origina porque el motor V8 preserva las variables del entorno léxico superior para mantener la continuidad de ejecución de los callbacks internos. Al acumularse llamadas sucesivas en el bucle de eventos, se genera una cadena de contextos anidados que saturan dinámicamente el Heap con bloques de datos masivos (*datosMasivos*), los cuales se vuelven inalcanzables pero imposibles de barrer por el recolector de basura [5].

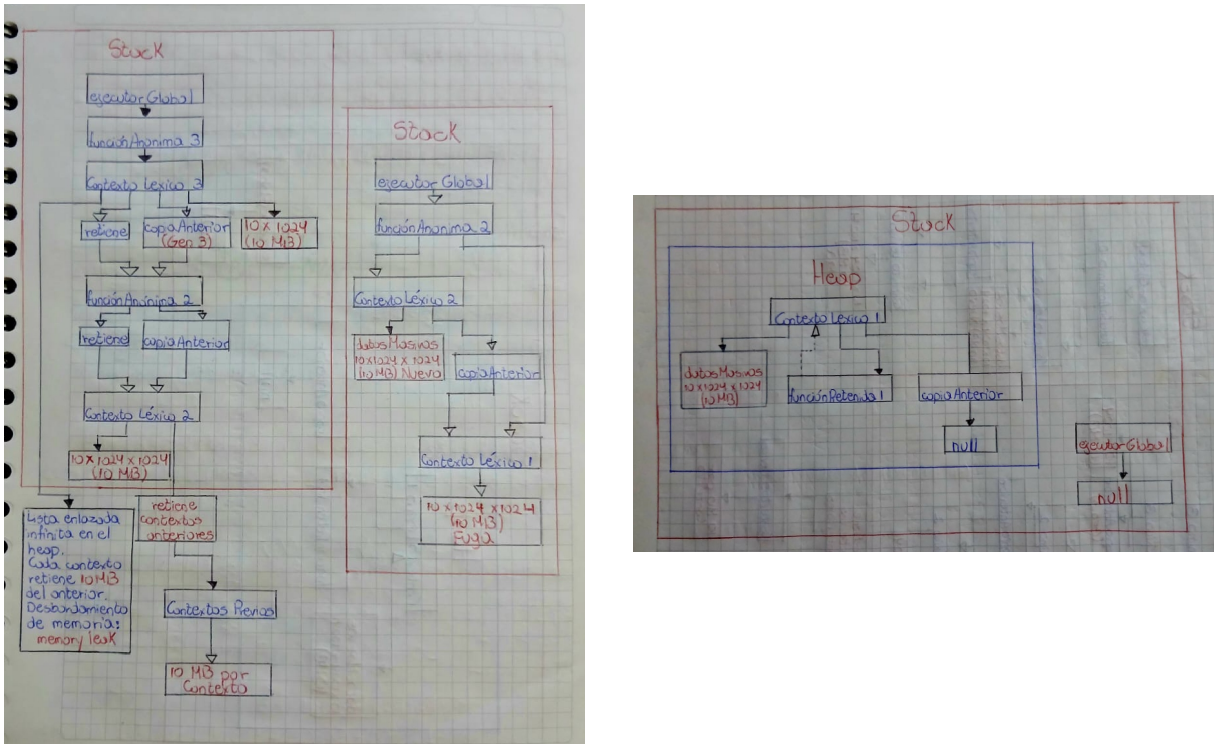


Figura 1: Diagramas hechos a mano (Partes 2 y 1) para Shared Lexical Context Leak.

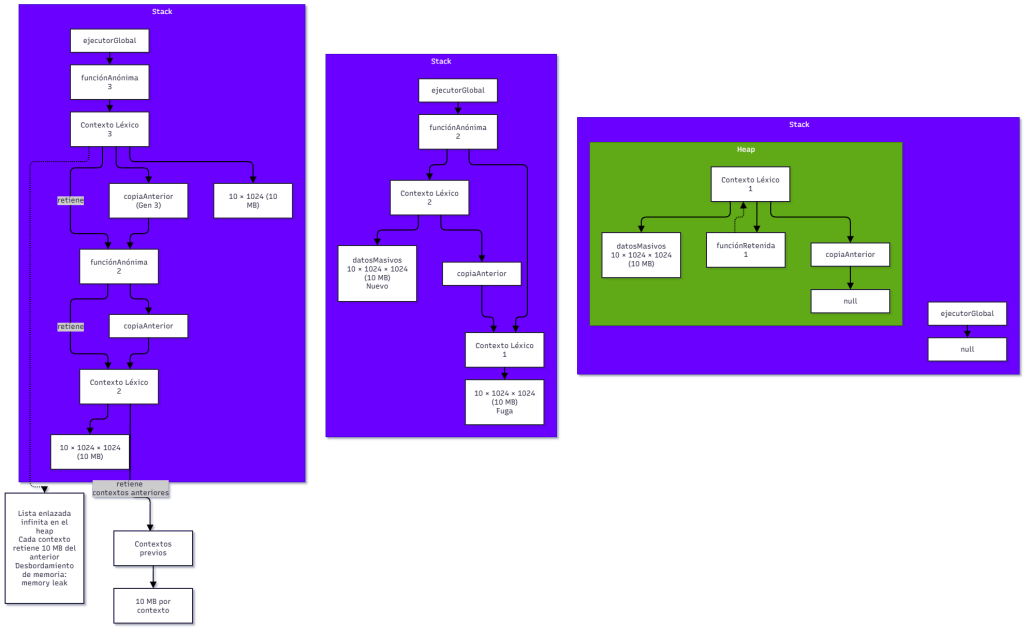


Figura 2: Diagrama digital de estados de memoria para Shared Lexical Context Leak.

4.2. Caso de Estudio 2: Grafo Circular y Referencias Débiles (Map vs WeakMap)

Descripción del problema: Se genera cuando múltiples objetos de tipo nodo se apuntan entre sí de forma bidireccional, creando una isla de referencias circulares aisladas dentro del Heap [4]. El conflicto crítico surge al almacenar la referencia raíz en una estructura de almacenamiento tradicional como un objeto *Map*, el cual mantiene vínculos y referencias fuertes de tipo operativo [4]. Incluso si se elimina la variable de entrada principal o se invoca la limpieza explícita de la clave, los nodos conservan punteros mutuos activos que impiden al algoritmo de rastreo (*Mark-and-Sweep*) determinar la inalcanzabilidad de la estructura [5]. Para evitar este comportamiento de retención estricta y permitir una liberación automática por parte del recolector de basura cuando los objetos pierden su contexto de uso activo, resulta indispensable migrar el diseño hacia referencias débiles mediante un *WeakMap* [5].

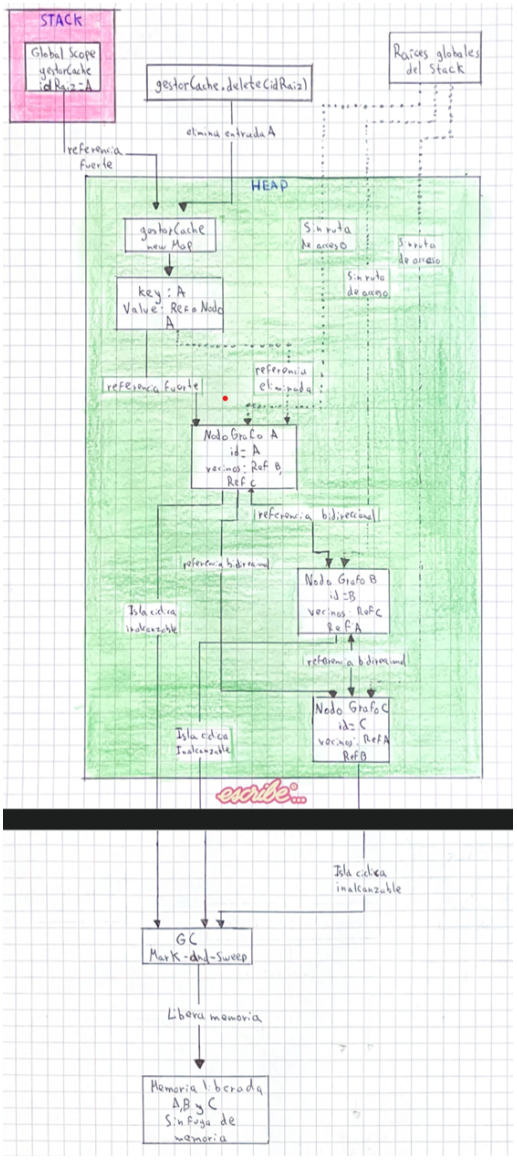


Figura 3: Diagrama hecho a mano para Grafo Circular y Referencias Débiles.

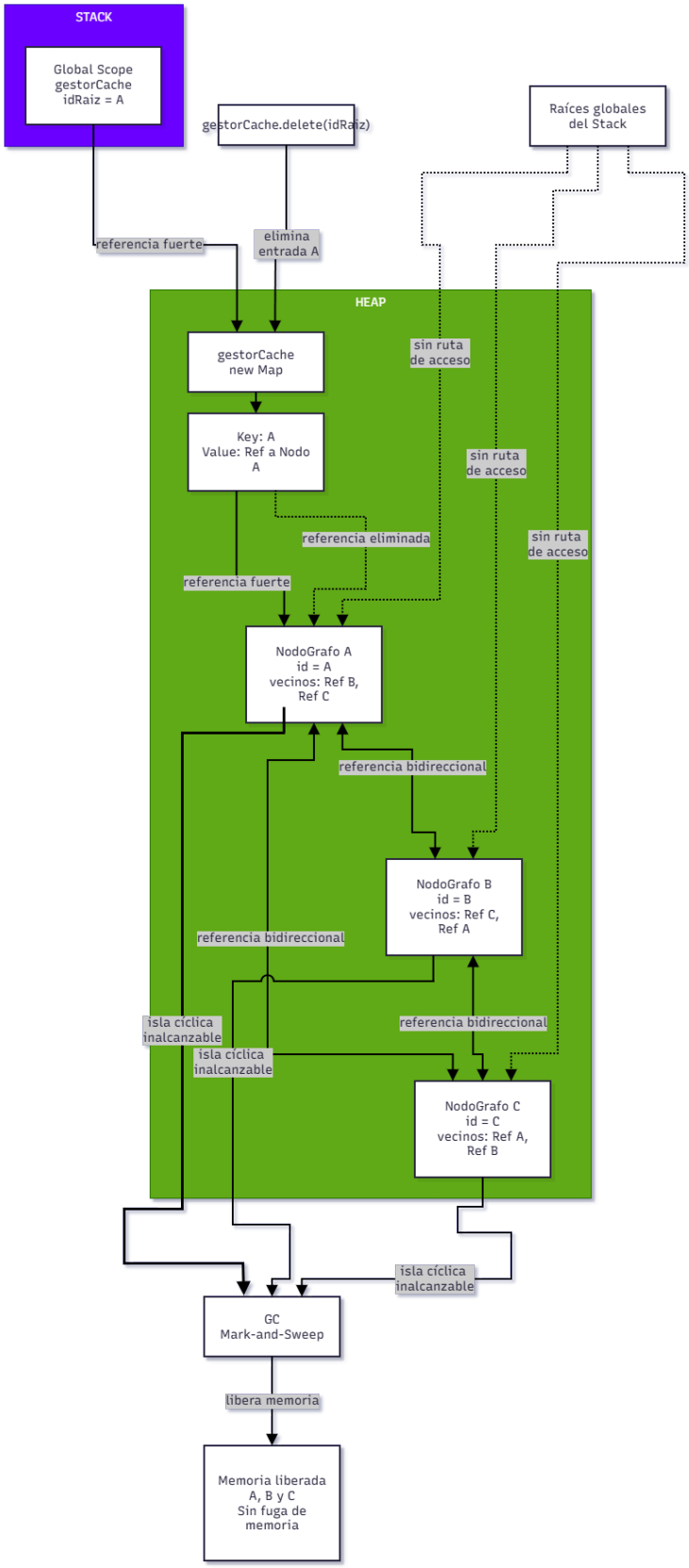


Figura 4: Diagrama digital de estados de memoria en Grafo Circular y referencias.

4.3. Caso de Estudio 3: Event Loop Queue & Retención por Callbacks

Descripción del problema: Se presenta cuando una instancia de objeto se inscribe en un emisor de eventos global (*Event Emitter*) [3]. Aunque la variable local salga del alcance del Stack, el *callback* registrado en la cola del *Event Loop* mantiene vivo al objeto completo en el Heap [3].

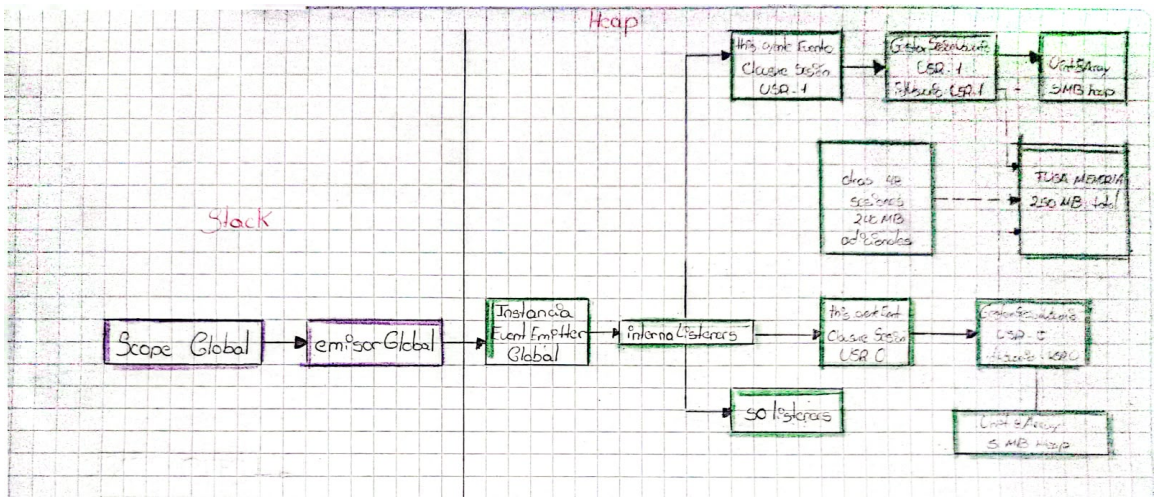


Figura 5: Diagrama hecho a mano para Event Loop Queue y Retención por Callbacks.

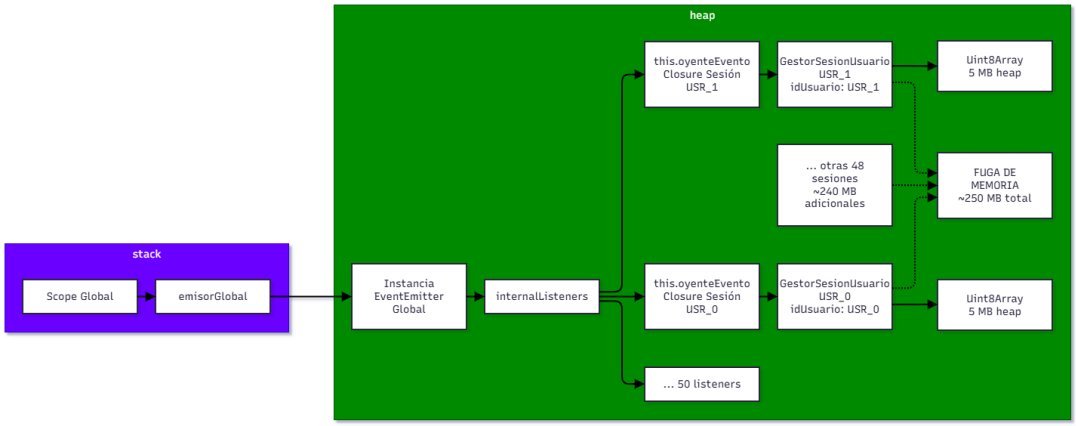


Figura 6: Diagrama digital de estados de memoria por retención en Event Loop.

Nota: Para ver los ejemplos de los diagramas de forma interactiva, revisa el repositorio: https://github.com/freddyordonez/Portafolio-de-Estructura-de-Datos



5. Conclusiones

- **C01: Jerarquía y coexistencia de la memoria:** El análisis detallado de los esquemas demuestra que la gestión eficiente de un programa depende críticamente de diferenciar las responsabilidades del Stack —administrando de forma rápida y automática los registros de activación locales bajo un modelo LIFO [1,2]— frente al Heap, que soporta la instanciación dinámica de objetos pesados y requiere un control estricto de sus referencias.
- **C02: Impacto visual del modelado en la detección de fugas:** La diagramación de estados de memoria se establece como una herramienta indispensable en la ingeniería de software moderna para visualizar relaciones complejas, tales como grafos cíclicos o cierres léxicos (*closures*), permitiendo identificar con precisión quirúrgica aquellas referencias activas u ocultas que retienen datos de forma indebida en el sistema.
- **C03: Optimización y robustez en entornos dinámicos:** Los casos de estudio evidencian que los errores comunes de arquitectura —como la retención inadvertida de *callbacks* en el Event Loop o el uso de estructuras de almacenamiento rígidas como *Map* en lugar de *WeakMap*— generan fugas de memoria críticas, haciendo indispensable el modelado gráfico para garantizar un diseño de software escalable y optimizado.

NOTA: El documento de ensayo se mantiene dentro del rango establecido y cumple con los estándares rigurosos de análisis técnico y visualización de arquitecturas de memoria.

Referencias

[1] P. W. Bible y L. Moser, *An Open Guide to Data Structures and Algorithms*. PALNI Open Press, 2023.

[2] OpenDSA, *Pointers and Memory Management Tutorials*. Virginia Tech, 2021.

[3] Mozilla Developer Network (MDN), "Memory Management," *JavaScript Guide*. [En línea]. Disponible: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Memory_management. [Accedido: Oct. 18, 2026].

[4] A. Mishra, "Understanding Stack vs Heap in JavaScript: Primitives and Non-Primitives," *DEV Community*, 2023. [En línea]. Disponible: https://dev.to/abhishek_mishra_2002/understanding-stack-vs-heap-in-javascript-primitives-and-non-primitives-jh. [Accedido: Oct. 18, 2026].

[5] F. Gerschau, "JavaScript Memory Management: How it works and how to prevent memory leaks," 2020. [En línea]. Disponible: https://felixgerschau.com/javascript-memory-management/. [Accedido: Oct. 18, 2026].

[6] Departamento de Ciencias de la Computación (DCC), "Punteros," *Wiki del curso CC3301*, Universidad de Chile. [En línea]. Disponible: https://wiki.dcc.uchile.cl/cc3301/punteros. [Accedido: Oct. 18, 2026].